

AdvR 07 – Conditions

Signalling Conditions

Three Types of Conditions

R has three conditions you can signal in code:

Condition	Function	Severity	Prefix
Error	<code>stop()</code> / <code>abort()</code>	Fatal	Error
Warning	<code>warning()</code> / <code>warn()</code>	Recoverable	Warning
Message	<code>message()</code> / <code>inform()</code>	Informational	(none)

```
stop("This is an error")
```

```
#> Error: This is an error
```

```
warning("This is a warning")
```

```
message("This is a message")
```

Use `call. = FALSE` to suppress the call in errors (best practice):

```
h <- function() stop("Bad!", call. = FALSE)
h()
```

```
#> Error: Bad!
```

`rlang::abort()` suppresses the call by default and stores a backtrace:

```
h <- function() abort("Bad!")
h()
```

```
#> Error in `h()`:
```

```
#> ! Bad!
```

Warnings

Warnings are weaker than errors — execution continues:

```
fw <- function() {  
  cat("1\n"); warning("W1")  
  cat("2\n"); warning("W2")  
}  
fw()  
#> 1  
#> 2  
#> Warning messages:  
#> 1: In fw() : W1  
#> 2: In fw() : W2
```

Control with options(warn = ...):

- warn = 0 (default): cached, displayed at end
- warn = 1: displayed immediately
- warn = 2: **promoted to errors** (useful for debugging)

Messages

Messages are for informational side-effects:

```
fm <- function() {  
  cat("1\n"); message("M1")  
  cat("2\n"); message("M2")  
}  
fm()
```

```
#> 1
```

```
#> 2
```

`cat()` **vs.** `message()`: Use `cat()` when printing is the *purpose*. Use `message()` when it's a *side-effect* (user can suppress with `suppressMessages()`).

Exercises (Signalling)

- ❶ What does `appendLF` in `message()` do? How does it relate to `cat()`?
- ❷ Write a wrapper around `file.remove()` that throws an error if the file doesn't exist.

Solutions (Signalling)

- 1 appendLF = TRUE (default) appends a newline \n to the message. cat() never appends a newline — you must add "\n" explicitly.

2

```
file_remove_strict <- function(path) {  
  if (!file.exists(path)) {  
    stop("Can't delete \"", path,  
        "\": doesn't exist.", call. = FALSE)  
  }  
  file.remove(path)  
}  
file_remove_strict("nonexistent.csv")
```

```
#> Error: Can't delete "nonexistent.csv": doesn't exist.
```


Ignoring Conditions

Suppressing Conditions

Three heavy-handed tools:

```
# Ignore errors --- execution continues  
try(log("x"))
```

```
#> Error in log("x") : non-numeric argument to mathematical function
```

```
# Ignore warnings  
suppressWarnings(as.numeric(c("1", "a")))
```

```
#> [1] 1 NA
```

```
# Ignore messages  
suppressMessages(message("Hello"))
```

Assignment inside try() — set a default on failure:

```
default <- NULL  
try(default <- read.csv("maybe-bad.csv"), silent = TRUE)
```

Handling Conditions

Condition Objects

Conditions are S3 objects with message and call components:

```
cnd <- catch_cnd(stop("An error"))  
str(cnd)
```

```
#> List of 2  
#> $ message: chr "An error"  
#> $ call    : language force(expr)  
#> - attr(*, "class")= chr [1:3] "simpleError" "error" "condition"
```

`rlang::catch_cnd()` captures a condition object for inspection. `conditionMessage(cnd)` extracts the message text.

tryCatch() — Exiting Handlers

`tryCatch()` registers **exiting** handlers — control leaves the protected code:

```
f3 <- function(x) {  
  tryCatch(  
    error = function(cnd) NA,  
    log(x)  
  )  
}  
f3("x")
```

```
#> [1] NA
```

The handler **exits** the block — remaining code is skipped:

```
tryCatch(  
  message = function(cnd) "Caught!",  
  {  
    message("Here")  
    stop("Never reached!")  
  }  
)
```

```
#> [1] "Caught!"
```

tryCatch() Details

Extract the error message inside the handler:

```
tryCatch(  
  error = function(cnd) {  
    paste0("--", conditionMessage(cnd), "--")  
  },  
  stop("This is an error")  
)
```

```
#> [1] "--This is an error--"
```

finally runs cleanup code regardless of success/failure:

```
path <- tempfile()  
tryCatch(  
  { writeLines("Hi!", path) },  
  finally = { unlink(path) }  
)
```

Calling handlers: code execution **continues** after the handler:

```
withCallingHandlers(  
  message = function(cnd) cat("Caught!\n"),  
  {  
    message("Someone there?")  
    message("Why, yes!")  
  }  
)
```

```
#> Caught!
```

```
#> Caught!
```

Compare: `tryCatch()` exits on first catch. `withCallingHandlers()` catches but **keeps going**.

Muffling Conditions

Use `cnd_muffle()` to prevent a condition from propagating:

```
withCallingHandlers(  
  message = function(cnd) {  
    cat("Caught:", conditionMessage(cnd))  
    cnd_muffle(cnd)  
  },  
  message("Hello")  
)
```

```
#> Caught: Hello
```

Without muffling, the condition bubbles up to the default handler (which prints it).

Exercises (Handling)

- 1 Predict the results:

```
show_condition <- function(code) {  
  tryCatch(  
    error = function(cnd) "error",  
    warning = function(cnd) "warning",  
    message = function(cnd) "message",  
    { code; NULL }  
  )  
}  
  
show_condition(stop("!"))  
show_condition(10)  
show_condition(warning("?!"))  
show_condition({10; message("?"); warning("?!" )})
```

- 2 Explain the output:

```
withCallingHandlers(  
  message = function(cnd) message("b"),  
  withCallingHandlers(  
    message = function(cnd) message("a"),  
    message("c")  
  )  
)
```

① Results:

```
show_condition(stop("!"))      # "error"
```

```
#> [1] "error"
```

```
show_condition(10)           # NULL (no condition)
```

```
#> NULL
```

```
show_condition(warning("?!" )) # "warning"
```

```
#> [1] "warning"
```

```
show_condition({10; message("?"); warning("?!" )}) # "message"
```

```
#> [1] "message"
```

The last returns "message" because `tryCatch()` **exits** on the first caught condition. The `warning("?!" )` is never reached.

- ② `message("c")` triggers inner handler → calls `message("a")` → outer handler catches it → calls `message("b")` → not caught, prints **b**. Then "a" propagates to default → prints **a**. Original "c" bubbles up to outer handler → calls `message("b")` again → prints **b**. Then "c" reaches default → prints **c**.

Output: b, a, b, c.

Custom Conditions

Why Custom Conditions?

Built-in conditions only have a text message. Matching errors by string is fragile:

```
log(letters)
```

```
#> Error in log(letters): non-numeric argument to mathematical function
```

Better: attach **metadata** so handlers can dispatch on structure, not text.

Creating Custom Conditions with abort()

`rlang::abort()` accepts a subclass and extra fields:

```
abort_bad_argument <- function(arg, must, not = NULL) {  
  msg <- glue::glue("`{arg}` must {must}")  
  if (!is.null(not)) {  
    msg <- glue::glue("{msg}; not {typeof(not)}.")  
  }  
  abort("error_bad_argument",  
    message = msg, arg = arg, must = must, not = not  
  )  
}  
abort_bad_argument("x", "be numeric", not = "a")
```

```
#> Error in `abort_bad_argument()`:  
#> ! `x` must be numeric; not character.
```

Handling Custom Conditions

Use the custom class in `tryCatch()` for targeted handling:

```
my_log <- function(x, base = exp(1)) {  
  if (!is.numeric(x))  
    abort_bad_argument("x", "be numeric", not = x)  
  base::log(x, base = base)  
}  
  
tryCatch(  
  error_bad_argument = function(cnd) "bad_argument",  
  error = function(cnd) "other error",  
  my_log("a")  
)
```

```
#> [1] "bad_argument"
```

Order matters: put the most specific handler first. `error` would catch everything before `error_bad_argument`.

Applications

Useful tryCatch() Patterns

Default on failure:

```
fail_with <- function(expr, value = NULL) {  
  tryCatch(error = function(cnd) value, expr)  
}  
fail_with(log(10), NA_real_)
```

```
#> [1] 2.302585
```

```
fail_with(log("x"), NA_real_)
```

```
#> [1] NA
```

More tryCatch() Patterns

Does it error?

```
does_error <- function(expr) {  
  tryCatch(error = function(cnd) TRUE,  
    { expr; FALSE })  
}  
does_error(1 + 1)
```

```
#> [1] FALSE
```

```
does_error(stop("!"))
```

```
#> [1] TRUE
```

Safe Execution

Safe result + error (related to `purrr::safely()`):

```
safety <- function(expr) {  
  tryCatch(  
    error = function(cnd) list(result = NULL, error = cnd),  
    list(result = expr, error = NULL)  
  )  
}  
str(safety(1 + 10))
```

```
#> List of 2  
#> $ result: num 11  
#> $ error : NULL
```

Promote warnings to errors (useful for debugging):

```
warning2error <- function(expr) {  
  withCallingHandlers(  
    warning = function(cnd) abort(conditionMessage(cnd)),  
    expr  
  )  
}
```

- 1 How could you rewrite `show_condition()` to use a **single** handler?

Solutions (Applications)

- 1 Use the condition class (parent of all conditions):

```
show_condition2 <- function(code) {  
  tryCatch(  
    condition = function(cnd) {  
      if (inherits(cnd, "error"))   return("error")  
      if (inherits(cnd, "warning")) return("warning")  
      if (inherits(cnd, "message")) return("message")  
    },  
    { code; NULL }  
  )  
}  
show_condition2(stop("!"))
```

```
#> [1] "error"
```

```
show_condition2(warning("?"))
```

```
#> [1] "warning"
```